

学号： 2006410127

大数据技术基础第二次作业

班级：计算机 2101

姓名：袁雄杰

作业五：抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

1.代码：

```
# 用于爬取信息

import requests

# 用于解析网页

from bs4 import BeautifulSoup

# 用于正则匹配找到目标项目

import re

# 对 csv 文件的操作

import csv

# 打开文件

# a+权限追加写入

# newline="" 用于取消自动换行

fp = open("data.csv", "a+", newline="")

# 修饰，处理成支持 csv 读取的文件

csv_fp = csv.writer(fp)

# 设置 csv 文件内标题头

head = ['日期', '最高气温', '最低气温']

# 写入标题

csv_fp.writerow(head)
```

```
# UA 伪装

headers = {

    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:86.0)

Gecko/20100101 Firefox/86.0"

}

# 存放全部数据

data = []

# 进行 url 拼接，主要拼接的是年份和月份

# 从 2021 年到 2022 年

for i in range(2021, 2022):

    # 从 1 月到 12 月

    for j in range(1, 13):

        # 字符串化

        i = str(i)

        # 小于 10 则补 0

        if j < 10:

            j = "0" + str(j)

        else:

            # 字符串化

            j = str(j)

        # 完成拼接

        url = "http://www.tianqihoubao.com/lishi/beijing/month/" +
```

```
i + j + ".html"
```

```
# 获取响应
```

```
response = requests.get(url=url, headers=headers)
```

```
# 设置编码为 gbk
```

```
response.encoding = 'gbk'
```

```
# 获取响应文本数据
```

```
page = response.text
```

```
# 用 BeautifulSoup 解析网页
```

```
soup = BeautifulSoup(page, 'lxml')
```

```
# 获取所有 tr 标签
```

```
tr_list = soup.find_all('tr')
```

```
# 解析每一个 tr 标签
```

```
for tr in tr_list:
```

```
    # 用于存放一天的数据
```

```
    one_day = []
```

```
    # 字符串化便于正则匹配
```

```
    tr = str(tr)
```

```
    # 去除所有空格
```

```
    tr = tr.replace(" ", "")
```

```
    # 取出日期
```

```
    date = re.findall(r'title="(.*?)北京天气预报">', tr)
```

```
    # 如果取到则放入 one——day 存放
```

```

if date:

    one_day.append(date[0])

# 取出最高温和最低温

tem = re.findall(r'(.*)°C', tr)

# 如果取到则放入 one——day 存放

if tem:

    one_day.append(tem[0])

    one_day.append(tem[1])

# 如果完整的取到一天的数据则放入 data 存放

if len(one_day) == 3:

    data.append(one_day)

    print(one_day)

    # 写入 csv 文件

    csv_fp.writerow(one_day)

# 关闭文件指针

fp.close()

```

2.运行截图：

日期	最高气温	最低气温
2021年1月1日	-11	0
2021年1月2日	-9	1
2021年1月3日	-9	-1
2021年1月4日	-11	-1
2021年1月5日	-11	-2
2021年1月6日	-17	-12
2021年1月7日	-14	-7
2021年1月8日	-10	-3
2021年1月9日	-8	2
2021年1月10日	-7	2
2021年1月11日	-8	2
2021年1月12日	-7	5
2021年1月13日	-6	8
2021年1月14日	-4	4
2021年1月15日	-8	4
2021年1月16日	-8	0
2021年1月17日	-7	4
2021年1月18日	-6	4
2021年1月19日	-7	-2
2021年1月20日	-6	3
2021年1月21日	-5	6
2021年1月22日	-7	5
2021年1月23日	-6	4
2021年1月24日	-5	5
2021年1月25日	-6	3
2021年1月26日	-7	8
2021年1月27日	-6	4
2021年1月28日	-9	1

作业六：葡萄酒数据集（wine.data）搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

1. 代码：

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

# 读取数据集

data = pd.read_csv('wine.data', header=None)

# 将数据集分为特征和标签

X = data.iloc[:, 1:]

y = data.iloc[:, 0]

from sklearn.preprocessing import LabelEncoder

#XGBoost 要求目标变量必须是从 0 开始的整数, 而 y 变量

取值是 [1, 2, 3],使

用 LabelEncoder 进行转换

le = LabelEncoder()

xgy = le.fit_transform(y)
```

```
# 将数据集分为训练集和测试集

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy,
test_size=0.3, random_state=42)

# 建立决策树模型

dtc = DecisionTreeClassifier(random_state=42)

dtc.fit(X_train, y_train)

dtc_score = dtc.score(X_test, y_test)

# 建立随机森林模型

rfc = RandomForestClassifier(random_state=42)

rfc.fit(X_train, y_train)

rfc_score = rfc.score(X_test, y_test)

# 建立 xgBoost 模型

xgb = XGBClassifier(random_state=42)

xgb.fit(X_train, xgy_train)

xgb_score = xgb.score(X_test, xgy_test)

# 输出各个模型在测试集上的表现

print('Decision Tree Classifier score:', dtc_score)

print('Random Forest Classifier score:', rfc_score)

print('XGBoost Classifier score:', xgb_score)
```

2. 运行截图：

```
In [1]: runfile('D:/python/作业/新建文件夹/wine/5.py', wdi
Decision Tree Classifier score: 0.9629629629629629
Random Forest Classifier score: 1.0
XGBoost Classifier score: 0.9629629629629629
```

This version of python seems to be incorrectly compiled
(internal generated filenames are not absolute).
This may make the debugger miss breakpoints.
Related bug: <http://bugs.python.org/issue1666807>